

Arreglos Tridimensionales (Listas de Listas de Listas)

- Concepto: Python permite anidar listas a cualquier profundidad. Un arreglo 3D sirve para organizar datos complejos multidimensionales.

Definición e invocación de funciones

- Propósito: Permiten reutilizar código repetido. Un cambio realizado dentro de la función se propaga automáticamente a todos los lugares donde se invoca.

Sintaxis básicas:

```
def nombre_de_funcion():  
    #Cuerpo de la función
```

Reglas clave

1. Debe definirse con la palabra reservada `def`, seguida del nombre, paréntesis `()` y dos puntos `:`.
2. Orden de lectura: Las funciones deben definirse antes de ser invocadas; de lo contrario, Python lanzará un `Name Error`.
3. Conflictos de nombres: una variable y una función no pueden compartir el mismo nombre en el mismo ámbito.

Parámetros y Argumentos

Diferencia

- Parámetros: variables definidas entre los paréntesis de la función que solo existen dentro de su cuerpo.

- Argumentos: valores reales que se envían a la función al momento de invocarla.

Formas de Pasar Argumentos

- Posicionales: Se asignan a los parámetros en el orden establecido en que se definen (ej. `f(a, b)`).

- Con palabra clave (keywords): se especifica el nombre del parámetro explícitamente (ej. `f(b=2, a=1)`).

edube.org/learn/python-essentials-1-esp-edube/laboratorio-un-a-ntilde-o-bisiesto-escribiendo-tus-propias-funciones-2

Python

4.3.1.6 LABORATORIO: Un año bisiesto: escribiendo tus propias funciones

LABORATORIO

Tiempo Estimado

10-15 minutos

Nivel de Dificultad

Fácil

Objetivos

Familiarizar al estudiante con:

- Proyectar y escribir funciones con parámetros.
- Utilizar la instrucción return.
- Probar las funciones.

Escenario

Tu tarea es escribir y probar una función que tome un argumento (un año) y devuelva: True si el año es un año bisiesto, o False si no lo es.

Parte del esqueleto de la función ya está en el editor.

Nota: también hemos preparado un breve código de prueba, que puedes utilizar para probar tu función.

El código utiliza dos listas: una con los datos de prueba y la otra con los resultados esperados. El código te dirá si alguno de tus resultados no es válido.

```
1- def is_year_leap(year):
2-     if year % 4 != 0:
3-         return False
4-     elif year % 100 != 0:
5-         return True
6-     elif year % 400 != 0:
7-         return False
8-     else:
9-         return True
10-
11- test_years = [1900, 2000, 2016, 1987]
12- test_results = [False, True, True, False]
13- for i in range(len(test_years)):
14-     yr = test_years[i]
15-     print(yr, "<--> ", end="")
16-     result = is_year_leap(yr)
17-     if result == test_results[i]:
18-         print("OK")
19-     else:
20-         print("Fallido")
21-
22-
```

Console 1

1900 -->OK
2000 -->OK
2016 -->OK
1987 -->OK

PrevNext

edube.org/learn/python-essentials-1-esp-edube/laboratorio-quest-cu-acute-ntos-d-iacute-as-escribiendo-y-utilizando-tus-propias-funciones-2

Python

4.3.1.7 LABORATORIO: ¿Cuántos días?: escribiendo y utilizando tus propias funciones

LABORATORIO

Tiempo Estimado

15-20 minutos

Nivel de Dificultad

Medio

Requisitos Previos

LABORATORIO 4.1.3.6

Objetivos

Familiarizar al estudiante con:

- Proyectar y escribir funciones parametrizadas.
- Utilizar la instrucción return.
- Utilizar las funciones propias del estudiante.

Escenario

Tu tarea es escribir y probar una función que tome dos argumentos (un año y un mes) y devuelva el número de días del mes/año dado (mientras que solo febrero es sensible al valor 'year', tu función deberá ser universal).

La parte inicial de la función está lista. Ahora, haz que la función devuelva True si los argumentos no tienen sentido.

Por supuesto, puedes (y debes) utilizar la función previamente escrita y probada (LABORATORIO 4.1.3.6). Puede ser muy útil. Te recomendamos que utilices una lista con los meses. Puedes crearla dentro de la función; esta truco acortará significativamente el código.

Hemos preparado un código de prueba. Ampliálo para incluir más casos de prueba.

```
1- def is_year_leap(year):
2-     if year % 4 != 0:
3-         return False
4-     elif year % 100 != 0:
5-         return True
6-     elif year % 400 != 0:
7-         return False
8-     else:
9-         return True
10-
11- def days_in_month(year, month):
12-     if year < 1 or month < 1 or month > 12:
13-         return None
14-
15-     days_per_month = (31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31)
16-
17-     if month == 2 and is_year_leap(year):
18-         return 29
19-
20-     return days_per_month[month - 1]
21-
22- test_years = [1900, 2000, 2016, 1987]
23- test_months = [12, 2, 1, 11]
24- test_results = [(28, 29, 31, 30)]
25- for i in range(len(test_years)):
26-     yr = test_years[i]
27-     mo = test_months[i]
28-     print(yr, mo, "<--> ", end="")
29-     result = days_in_month(yr, mo)
30-     if result == test_results[i]:
31-         print("OK")
32-     else:
33-         print("Fallido")
34-
35-
```

Console 1

1900 2 -->OK
2000 2 -->OK
2016 1 -->OK
1987 11 -->OK

PrevNext

edube.org/learn/python-essentials-1-esp-edube/laboratorio-d-iacute-a-del-a-ntilde-o-escribiendo-y-utilizando-tus-propias-funciones-2

Python

4.3.1.8 LABORATORIO: Día del año: escribiendo y utilizando tus propias funciones

LABORATORIO

Tiempo Estimado

20-30 minutos

Nivel de Dificultad

Medio

Requisitos Previos

LABORATORIO 4.1.3.6

LABORATORIO 4.1.3.7

Objetivos

Familiarizar al estudiante con:

- Proyectar y escribir funciones con parámetros.
- Utilizar la sentencia return.
- Construir un conjunto de funciones de utilidad.
- Utilizar las funciones propias del estudiante.

Escenario

Tu tarea es escribir y probar una función que tome tres argumentos (un año, un mes y un día del mes) y devuelva el día correspondiente del año, o devuelva None si cualquiera de los argumentos no es válido.

Debes utilizar las funciones previamente escritas y probadas. Agrega algunos casos de prueba al código. Esta prueba es solo el comienzo.

```
1- def is_year_leap(year):
2-     if year % 4 != 0:
3-         return False
4-     elif year % 100 != 0:
5-         return True
6-     elif year % 400 != 0:
7-         return False
8-     else:
9-         return True
10-
11- def days_in_month(year, month):
12-     if year < 1 or month < 1 or month > 12:
13-         return None
14-
15-     days_per_month = (31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31)
16-
17-     if month == 2 and is_year_leap(year):
18-         return 29
19-
20-     return days_per_month[month - 1]
21-
22- def day_of_year(year, month, day):
23-     if year < 1 or month < 1 or month > 12 or day < 1:
24-         return None
25-
26-     max_days = days_in_month(year, month)
27-     if day > max_days or day < 0:
28-         return None
29-
30-     total_days = day
31-     for m in range(1, month):
32-         total_days += days_in_month(year, m)
33-
34-     return total_days
35-
36- print(day_of_year(2000, 12, 31))
37-
```

Console 1

366

PrevNext

← → ↺

edube.org/learn/python-essentials-1-esp-edube/laboratorio-n-ucute-meros-primos-iquest-c-acute-mo-encontrarlos-2

🔍 ☆ ⬇️ J ⋮

Todos los marcadores

Python

INSTITUTE

4.3.1.9 LABORATORIO: Números primos: ¿Cómo encontrarlos?

🏠

Python

INSTITUTE

4.3.1.9 LABORATORIO: Números primos: ¿Cómo encontrarlos?

🏠

Nivel de Dificultad

Medio

Objetivos

- Familiarizar al estudiante con nociones y algoritmos clásicos.
- Mejorar las habilidades del estudiante para definir y emplear funciones.

Escenario

Un número natural es **primo** si es mayor que 1 y no tiene divisores más que 1 y sí mismo.

¿Complicado? De ninguna manera. Por ejemplo, 8 no es un número primo, ya que puedes dividirlo entre 2 y 4 (no podemos usar divisores iguales a 1 y 8, ya que la definición lo prohíbe).

Por otra parte, 7 es un número primo, ya que no podemos encontrar ningún divisor para él.

Tu tarea es escribir una función que verifique si un número es primo o no.

La función:

- Se llama `is_prime`.
- Tome un argumento (el valor a verificar).
- Devuelva `True` si el argumento es un número primo, y `False` de lo contrario.

Sugerencia: intenta dividir el argumento por todos los valores posteriores (comenzando desde 2) y verifica el resto: si es cero, tu número no puede ser un número primo; analiza cuidadosamente cuándo deberías detener el proceso.

Si necesitas conocer la raíz cuadrada de cualquier valor, puedes utilizar el operador `**`. Recuerda: la raíz cuadrada de x es lo mismo que $x^{0.5}$.

Completa el código en el editor.

Ejecuta tu código y verifica si tu salida es la misma que la nuestra.

Datos de prueba

Salida esperada:

2 3 5 7 11 13 17 19

1

def is_prime(num):

2 if num < 2:

3 return False

4 for i in range(2, int(num ** 0.5) + 1):

5 if num % i == 0:

6 return False

7 return True

8

9 for i in range(1, 20):

10 if is_prime(i):

11 print(i, end=" ")

12 print()

Console 1

2 3 5 7 11 13 17 19

Prev

Next

← → ↺

edube.org/learn/python-essentials-1-esp-edube/lab-convirtiendo-el-consumo-de-combustible-2

🔍 ☆ ⬇️ J ⋮

Todos los marcadores

Python

INSTITUTE

4.3.1.10 LAB: Convirtiendo el consumo de combustible

🏠

Python

INSTITUTE

4.3.1.10 LAB: Convirtiendo el consumo de combustible

🏠

Tiempo estimado

10-15 minutos

Nivel de Dificultad

Fácil

Objetivos

- Mejorar las habilidades del estudiante para definir, utilizar y probar funciones.

Escenario

El consumo de combustible de un automóvil se puede expresar de muchas maneras diferentes. Por ejemplo, en Europa, se muestra como la cantidad de combustible consumido por cada 100 kilómetros.

En los EE. UU., se muestra como la cantidad de millas recorridas por un automóvil con un galón de combustible.

Tu tarea es escribir un par de funciones que conviertan l/100km a mpg (millas por galón), y viceversa.

Las funciones:

- Se llaman `liters_100km_to_miles_gallon` y `miles_gallon_to_liters_100km` respectivamente.
- Toman un argumento (el valor correspondiente a sus nombres).

Completa el código en el editor.

Ejecuta tu código y verifica si tu salida es la misma que la nuestra.

Aquí hay información para ayudarte:

- 1 milla = 1609.344 metros.
- 1 galón = 3.785411784 litros.

Salida esperada:

60.311431423393162

81.36136444444444

23.52145833333333

3.300733367617447

7.4909122972339316

10.009131205679757

1

def liters_100km_to_miles_gallon(liters):

2 gallons = liters / 3.785411784

3 miles = 1609.344 / gallons

4 return miles / gallons

5

6 def miles_gallon_to_liters_100km(miles):

7 km = miles * 1609.344 / 1000

8 liters = 3.785411784

9 return liters / km * 100

10

11 print(liters_100km_to_miles_gallon(60.3))

12 print(liters_100km_to_miles_gallon(81.3))

13 print(liters_100km_to_miles_gallon(23.5))

14 print(miles_gallon_to_liters_100km(60.3))

15 print(miles_gallon_to_liters_100km(81.3))

16 print(miles_gallon_to_liters_100km(23.5))

17

Console 1

60.311431423393162

81.36136444444444

23.52145833333333

3.300733367617447

7.4909122972339316

10.009131205679757

Prev

Next

Arreglos Tridimensionales (Listas de Listas de Listas)

• Concepto: Python permite anidar listas a cualquier profundidad. Un arreglo 3D sirve para organizar datos complejos multidimensionales.

Definición e invocación de funciones

• Propósito: Permiten reutilizar código repetido. Un cambio realizado dentro de la función se propaga automáticamente a todos los lugares donde se invoca.

• Sintaxis básicas:

```
def nombre_de_funcion():  
    #Cuerpo de la función
```

Reglas clave

1. Debe definirse con la palabra reservada `def`, seguida del nombre. Parentesis `()` y dos puntos `:`.

2. Orden de lectura: Las funciones deben definirse antes de ser invocadas; de lo contrario, Python lanzará un `Name Error`.

3. Conflictos de nombres: una variable y una función no pueden compartir el mismo nombre en el mismo ámbito.

Parámetros y Argumentos

Diferencia

• Parámetros: variables definidas entre los parentesis de la función que solo existen dentro de su cuerpo.

• Argumentos: valores reales que se envían a la función al momento de invocarla.

Formas de Pasar Argumentos

• Posicionales: Se asignan a los parámetros en el orden establecido en que se envían (ej. `f(a,b)`).

• con palabra clave (keywords): se especifica el nombre del parámetro explícitamente (ej. `f(b=2, a=1)`).